

CAAJ - Documentación Completa del Proyecto

Descripción General

CAAJ es una plataforma web para compartir apuntes universitarios. Los usuarios pueden publicar apuntes de distintas materias, comentar, votar (like/dislike), y adjuntar archivos. El sistema tiene roles (alumno, profesor, admin) y funciona con Flask + MySQL.

Arquitectura del Proyecto

Estructura MVC (Modelo-Vista-Controlador)

```
CAAJ/  
├── run.py           # Punto de entrada de la aplicación  
├── route.py         # Define las rutas (URLs)  
├── controller.py    # Lógica de negocio  
├── model.py         # Interacción con la BD  
├── _mysql_db.py     # Conexión y operaciones SQL  
├── appConfig.py     # Configuración del proyecto  
├── templates/       # Vistas HTML (Jinja2)  
├── static/          # CSS, JS, imágenes  
│   ├── css/  
│   ├── js/  
│   ├── img/  
│   └── uploads/     # Archivos subidos por usuarios  
└── docs/  
    └── caaj.sql      # Esquema de la base de datos
```

Base de Datos MySQL

Esquema de Tablas

1. usuario

```
CREATE TABLE usuario (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  apodo VARCHAR(50) NOT NULL,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  pass VARCHAR(255) NOT NULL,  
  id_rol INT NOT NULL,  
  nombre VARCHAR(50) NOT NULL,  
  apellido VARCHAR(50) NOT NULL,  
  dni VARCHAR(20) NOT NULL,  
  FOREIGN KEY (id_rol) REFERENCES rol(id)  
);
```

Función: Almacena información de usuarios (alumnos, profesores, admins).

2. rol

```
CREATE TABLE rol (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  nombre VARCHAR(50) NOT NULL  
);
```

Valores:

- 1 = alumno
- 2 = profesor
- 3 = administrador

3. materia

```
CREATE TABLE materia (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  materia VARCHAR(100) NOT NULL,  
  UUID VARCHAR(32) NOT NULL  
);
```

Función: Catálogo de materias (Matemáticas, Física, Historia, etc.)

4. apunte

```
CREATE TABLE apunte (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  id_usuario INT NOT NULL,  
  id_materia INT NOT NULL,  
  head VARCHAR(100),           -- Título del apunte  
  content TEXT,                -- Contenido en Markdown  
  tags VARCHAR(200),          -- Tags separados por comas  
  fechahora DATETIME NOT NULL,  
  FOREIGN KEY (id_usuario) REFERENCES usuario(id),  
  FOREIGN KEY (id_materia) REFERENCES materia(id)  
);
```

Función: Almacena los apuntes publicados por usuarios.

5. comentario

```
CREATE TABLE comentario (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  id_apunte INT NOT NULL,  
  id_usuario INT NOT NULL,  
  content TEXT NOT NULL,  
  fechahora DATETIME NOT NULL,  
  FOREIGN KEY (id_apunte) REFERENCES apunte(id) ON DELETE CASCADE,  
  FOREIGN KEY (id_usuario) REFERENCES usuario(id)  
);
```

Función: Comentarios en apuntes.

6. reaccion

```
CREATE TABLE reaccion (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  id_apunte INT NOT NULL,  
  id_usuario INT NOT NULL,  
  valor ENUM('like', 'dislike') NOT NULL,  
  FOREIGN KEY (id_apunte) REFERENCES apunte(id) ON DELETE CASCADE,  
  FOREIGN KEY (id_usuario) REFERENCES usuario(id),  
  UNIQUE KEY unique_voto (id_apunte, id_usuario)  
);
```

Función: Almacena votos (like/dislike). Un usuario solo puede votar una vez por apunte.

7. media

```
CREATE TABLE media (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  id_apunte INT NOT NULL,  
  nombre VARCHAR(100), -- Nombre original del archivo  
  path VARCHAR(255), -- Ruta en el servidor  
  FOREIGN KEY (id_apunte) REFERENCES apunte(id) ON DELETE CASCADE  
);
```

Función: Archivos adjuntos (PDF, imágenes, etc.)

Flujo Backend → Frontend

1. Inicio de la Aplicación

run.py:

```

from flask import Flask
from route import route

def main():
    app = Flask(__name__, template_folder='templates', static_folder='static')
    app.config['SECRET_KEY'] = 'tu_clave_secreta'
    route(app) # Registra todas las rutas
    app.run('0.0.0.0', 5000, debug=True)

main()

```

2. Definición de Rutas (route.py)

Las rutas conectan URLs con funciones del controlador:

```

@app.route('/apunte')
def apunte():
    apunteid = request.args.get('apunte') # GET parámetro ?apunte=123
    param = {}
    return apunte_pagina(param, apunteid)

```

Explicación:

- `@app.route('/apunte')` → Cuando visitas `http://localhost:5000/apunte?apunte=5`
- `request.args.get('apunte')` → Extrae el ID del apunte de la URL
- `apunte_pagina(param, apunteid)` → Llama al controlador que prepara los datos

3. Controlador (controller.py)

El controlador maneja la lógica de negocio:

```

def apunte_pagina(param, apunteid):
    obtenerMenuHead(param) # Carga el menú del header
    obtenerApunteXid(param, apunteid) # Busca el apunte en la BD
    obtenerMediaXid(param, apunteid) # Busca archivos adjuntos
    obtenerComentarios(param, apunteid) # Busca comentarios

    # Si el usuario está logueado, trae su voto
    id_usuario = session.get('id_usuario')
    if id_usuario:
        obtenerVotoUsuario(param, apunteid, id_usuario)

    return render_template('apunte.html', param=param)

```

Flujo:

1. Prepara el diccionario `param` con todos los datos necesarios

2. Llama a funciones del modelo para consultar la BD
3. Renderiza el template HTML con los datos

4. Modelo (model.py)

El modelo se comunica con la base de datos:

```
def obtenerApunteXidDB(result, id):
    q = """
        SELECT id_usuario, id_materia, head, content, tags, fechahora
        FROM apunte
        WHERE id=%s
    """
    filas = selectDB(BASE, q, (id,))

    if filas and len(filas) > 0:
        fila = filas[0]
        result['apunte'] = {
            'id': id,
            'usuario': obtenerUsuarioNombreXid(fila[0]),
            'id_materia': fila[1],
            'titulo': fila[2],
            'contenido': fila[3],
            'tags': fila[4],
            'fecha': fila[5]
        }
    return result
```

Explicación:

- `q` → Query SQL preparado (con `%s` para prevenir SQL injection)
- `selectDB(BASE, q, (id,))` → Ejecuta la consulta de forma segura
- Los valores en tupla `(id,)` reemplazan los `%s` de manera segura
- El resultado se guarda en un diccionario Python

5. Conexión MySQL (_mysql_db.py)

Función `selectDB` (Consultas SELECT)

```
def selectDB(configDB, sql, param=None):
    mydb = conectarBD(configDB) # Conecta a MySQL

    cursor = mydb.cursor()
    if param:
        cursor.execute(sql, param) # Ejecuta con parámetros seguros
    else:
        cursor.execute(sql)

    resultado = cursor.fetchall() # Obtiene todas las filas
```

```
cursor.close()
cerrarBD(mydb)

return resultado # Lista de tuplas [(col1, col2), ...]
```

Función `insertDB` (Insertar datos)

```
def insertDB(configDB, sql, param):
    mydb = conectarBD(configDB)
    cursor = mydb.cursor()

    cursor.execute(sql, param)
    mydb.commit() # ¡IMPORTANTE! Confirma los cambios

    resultado = cursor.rowcount # Número de filas afectadas
    cursor.close()
    cerrarBD(mydb)

    return resultado
```

Función `insertDB_return_id` (Insertar y devolver ID)

```
def insertDB_return_id(configDB, sql, param):
    mydb = conectarBD(configDB)
    cursor = mydb.cursor()

    cursor.execute(sql, param)
    mydb.commit()

    id = cursor.lastrowid # ID auto-generado por MySQL
    cursor.close()
    cerrarBD(mydb)

    return id
```

Uso en `model.py`:

```
def crearComentario(id_apunte, id_usuario, comentario):
    q = """
        INSERT INTO comentario (id_apunte, id_usuario, content, fechahora)
        VALUES (%s, %s, %s, NOW())
    """
    id_comentario = insertDB_return_id(BASE, q, (id_apunte, id_usuario,
comentario))
    return id_comentario # Devuelve el ID del comentario creado
```

Autenticación y Sesiones

Login Flow

1. Usuario ingresa credenciales (templates/login.html)

```
<form action="/logging" method="POST">
  <input type="text" name="mail" required />
  <input type="password" name="password" required />
  <input type="submit" value="Ingresar" />
</form>
```

2. Route recibe el POST (route.py)

```
@app.route('/logging', methods=['POST'])
def logging():
    param = {}
    return logging_process(param, request)
```

3. Controller valida usuario (controller.py)

```
def crearSesion():
    mirequest = {}
    getRequest(mirequest) # Extrae mail y password del formulario

    dicUsuario = {}
    if obtenerUsuarioXEmailPass(dicUsuario, mirequest.get("mail"),
mirequest.get("password")):
        cargarSesion(dicUsuario) # ¡Usuario válido! Crear sesión
        return True
    return False
```

4. Model consulta la BD (model.py)

```
def obtenerUsuarioXEmailPass(result, email, passw):
    q = """
        SELECT id, apodo, email, pass, id_rol, nombre, apellido, dni
        FROM usuario
        WHERE email=%s AND pass=%s
    """
    filas = selectDB(BASE, q, (email, passw))

    if filas and len(filas) > 0:
        result['id'] = filas[0][0]
        result['apodo'] = filas[0][1]
```

```

        result['email'] = filas[0][2]
        result['id_rol'] = filas[0][4]
        # ... etc
        return True
    return False

```

5. Controller carga la sesión

```

def cargarSesion(dicUsuario):
    session['id_usuario'] = dicUsuario['id']
    session['username'] = dicUsuario['apodo']
    session['email'] = dicUsuario['email']
    session['rol'] = dicUsuario['id_rol']
    # La sesión se guarda automáticamente en cookies cifradas

```

Seguridad:

- Flask usa `SECRET_KEY` para cifrar las cookies de sesión
- Las contraseñas deberían hashear (actualmente están en texto plano - [△](#) MEJORAR)

Subida de Archivos

Flujo completo

1. Formulario HTML (templates/nuevoapunte.html)

```

<form action="/publicar" method="POST" enctype="multipart/form-data">
    <input type="file" name="archivo[]" multiple />
    <input type="submit" value="Publicar" />
</form>

```

2. Controller procesa archivos (controller.py)

```

def upload_file(diResult):
    UPLOAD_EXTENSIONS = ['.jpg', '.png', '.jpeg', '.pdf', '.doc', '.docx']
    MAX_CONTENT_LENGTH = 1024 * 1024 * 5 # 5MB máximo

    files = request.files.getlist('archivo[]')
    diResult['archivos'] = []

    for f in files:
        if f and f.filename:
            file_extension = os.path.splitext(f.filename)[1].lower()
            filename_unique = str(uuid4()) + file_extension # Nombre único
            path_filename = os.path.join(config['upload_folder'], filename_unique)

```



```

if file_extension in UPLOAD_EXTENSIONS:
    f.save(path_filename) # Guarda en static/uploads/
    diResult['archivos'].append({
        'file_name': f.filename,
        'file_name_new': filename_unique
    })

```

3. Model guarda en BD (model.py)

```

def crearMedia(id_apunte, file_name, nombre_uuid):
    q = """
        INSERT INTO media (id_apunte, nombre, path)
        VALUES (%s, %s, %s)
    """
    val = (id_apunte, file_name, '/static/uploads/' + nombre_uuid)
    insertDB(BASE, q, val)

```

Resultado:

- Archivo físico: `static/uploads/a1b2c3d4-e5f6-7890.pdf`
- BD: `nombre="informe.pdf", path="/static/uploads/a1b2c3d4-e5f6-7890.pdf"`

Sistema de Búsqueda

Búsqueda con normalización de tildes

```

def buscarGlobal(result, query):
    # Normaliza tildes para buscar mejor
    q = """
        SELECT 'materia' as tipo, m.id, m.materia as titulo
        FROM materia m
        WHERE LOWER(
            REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(
                m.materia, 'á', 'a'), 'é', 'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u')
            ) LIKE LOWER(
                REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(
                    %s, 'á', 'a'), 'é', 'e'), 'í', 'i'), 'ó', 'o'), 'ú', 'u')
                )
        UNION
        SELECT 'apunte' as tipo, a.id, a.head as titulo
        FROM apunte a
        WHERE LOWER(...) LIKE LOWER(...)
        LIMIT 10
    """
    search_term = f'%{query}%' # Busca coincidencias parciales
    filas = selectDB(BASE, q, (search_term, search_term, search_term))

```

¿Por qué tantos REPLACE?

- MySQL compara 'matemáticas' ≠ 'matematicas'
- Los REPLACE quitan tildes de ambos lados para que coincidan
- Ejemplo: Usuario busca "física" → encuentra "Física"

Sistema de Votos (Like/Dislike)

Lógica de Toggle

```
def votar_apunte_db(id_apunte, id_usuario, tipo):
    # 1. Chequeamos si el usuario ya votó
    q_check = "SELECT valor FROM reaccion WHERE id_apunte=%s AND id_usuario=%s"
    filas = selectDB(BASE, q_check, (id_apunte, id_usuario))

    voto_existente = filas[0][0] if filas else None

    # 2. Si vota lo mismo que ya tenía, quitamos el voto (toggle)
    if voto_existente == tipo:
        q_del = "DELETE FROM reaccion WHERE id_apunte=%s AND id_usuario=%s"
        deleteDB(BASE, q_del, (id_apunte, id_usuario))
        voto_actual = None # Voto removido
    else:
        # 3. Si había voto diferente, lo borramos primero
        if voto_existente:
            deleteDB(BASE, q_del, (id_apunte, id_usuario))

        # 4. Insertamos el voto nuevo
        q_ins = "INSERT INTO reaccion (id_apunte, id_usuario, valor) VALUES (%s, %s, %s)"
        insertDB(BASE, q_ins, (id_apunte, id_usuario, tipo))
        voto_actual = tipo

    # 5. Contamos los votos totales
    q_count = "SELECT valor, COUNT(*) FROM reaccion WHERE id_apunte=%s GROUP BY valor"
    filas = selectDB(BASE, q_count, (id_apunte,))

    likes = dislikes = 0
    for tipo_v, count in filas or []:
        if tipo_v == 'like':
            likes = count
        elif tipo_v == 'dislike':
            dislikes = count

    return True, likes, dislikes, voto_actual
```

Casos de uso:

1. Usuario sin voto + click Like → Inserta 'like'

2. Usuario con 'like' + click Like → Borra voto (toggle)
3. Usuario con 'like' + click Dislike → Borra 'like', inserta 'dislike'

Comunicación Frontend ↔ Backend

AJAX con Fetch API

Ejemplo: Votar un apunte (static/js/apunte.js)

```
function votarApunte(tipo) {
  const id_apunte = document.getElementById("id_apunte").value;

  fetch("/votar_apunte", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      id_apunte: id_apunte,
      tipo: tipo, // 'like' o 'dislike'
    }),
  })
  .then((response) => response.json())
  .then((data) => {
    if (data.success) {
      // Actualizar la UI sin recargar
      document.getElementById("like-count").textContent = data.likes;
      document.getElementById("dislike-count").textContent = data.dislikes;

      // Activar/desactivar botones
      const likeBtn = document.querySelector(".upvote");
      const dislikeBtn = document.querySelector(".downvote");

      likeBtn.classList.remove("active");
      dislikeBtn.classList.remove("active");

      if (data.voto_actual === "like") {
        likeBtn.classList.add("active");
      } else if (data.voto_actual === "dislike") {
        dislikeBtn.classList.add("active");
      }
    }
  });
}
```

Backend (controller.py)

```
@app.route('/votar_apunte', methods=['POST'])
def votar_apunte():
```

```
data = request.get_json() # Recibe JSON del fetch
id_apunte = data.get('id_apunte')
tipo = data.get('tipo')
id_usuario = session.get('id_usuario')

exito, likes, dislikes, voto_actual = votar_apunte_db(id_apunte, id_usuario,
tipo)

# Devuelve JSON al frontend
return jsonify({
    'success': exito,
    'likes': likes,
    'dislikes': dislikes,
    'voto_actual': voto_actual
})
```

🔧 Queries SQL Más Importantes

1. Buscar apuntes de una materia con búsqueda

```
SELECT a.id, a.head, a.content, a.tags, u.apodo, m.materia
FROM apunte AS a
INNER JOIN usuario AS u ON a.id_usuario = u.id
INNER JOIN materia AS m ON m.id = a.id_materia
WHERE m.id = %s
AND (
    LOWER(REPLACE(..., a.head, ...)) LIKE LOWER(REPLACE(..., %s, ...))
    OR LOWER(REPLACE(..., a.tags, ...)) LIKE LOWER(REPLACE(..., %s, ...))
    OR LOWER(REPLACE(..., u.apodo, ...)) LIKE LOWER(REPLACE(..., %s, ...))
)
```

¿Qué hace?

- **INNER JOIN**: Une tablas para obtener nombre de usuario y materia
- **WHERE m.id = %s**: Filtra por materia específica
- **AND (... LIKE ...)**: Busca en título, tags o nombre de usuario
- Normaliza tildes para búsqueda flexible

2. Obtener apuntes de un usuario con materia

```
SELECT a.id, a.id_usuario, a.id_materia, a.head, a.content, a.tags, a.fechahora,
m.materia
FROM apunte a
LEFT JOIN materia m ON a.id_materia = m.id
WHERE a.id_usuario = %s
ORDER BY a.fechahora DESC
```

¿Qué hace?

- **LEFT JOIN**: Trae la materia (o NULL si no existe)
- **ORDER BY fechahora DESC**: Los más recientes primero

3. Contar likes y dislikes

```
SELECT valor, COUNT(*)
FROM reaccion
WHERE id_apunte = %s
GROUP BY valor
```

Resultado:

valor	COUNT(*)
like	15
dislike	3

4. Buscar comentarios de un apunte con usuario

```
SELECT c.id, c.id_usuario, u.apodo, c.content, c.fechahora
FROM comentario AS c
INNER JOIN usuario u ON c.id_usuario = u.id
WHERE c.id_apunte = %s
ORDER BY c.fechahora ASC
```

Seguridad

Prevención de SQL Injection

✗ MALO (Vulnerable):

```
q = f"SELECT * FROM usuario WHERE email='{email}'"
```

Si `email = ''`; `DROP TABLE usuario; --` → ¡Destruye la tabla!

☑ BUENO (Seguro):

```
q = "SELECT * FROM usuario WHERE email=%s"
cursor.execute(q, (email,))
```

Los parámetros se escapan automáticamente.

Control de Permisos

```
def borrar_apunte_process():
    rol = session.get('rol')
    id_usuario = session.get('id_usuario')

    # Solo el dueño o admin puede borrar
    if rol == 3 or (apunte.get('usuario', {}).get('id') == id_usuario):
        borrarApunteDB(id_apunte)
    else:
        return "No autorizado"
```

Ejemplo Completo: Crear un Apunte

1. Usuario completa el formulario (templates/nuevoapunte.html)

```
<form action="/publicar" method="POST" enctype="multipart/form-data">
  <select name="materia">
    <option value="1">Matemáticas</option>
  </select>
  <input name="titulo" placeholder="Título" />
  <textarea name="contenido">Contenido...</textarea>
  <input name="tags" placeholder="tags, separados, por, comas" />
  <input type="file" name="archivo[]" multiple />
  <button type="submit">Publicar</button>
</form>
```

2. Route recibe el POST (route.py)

```
@app.route('/publicar', methods=['POST'])
def publicar():
    param = {}
    return publicar_process(request, param)
```

3. Controller procesa (controller.py)

```
def publicar_process(request, param):
    id_apunte = cargarApunte(request)
    if id_apunte:
        return redirect(f'/apunte?apunte={id_apunte}')
    else:
        param['error_msg'] = "Error al crear el apunte"
```

```

        return render_template('nuevoapunte.html', param=param)

def cargarApunte(request):
    mirequest = {}
    getRequest(mirequest) # Extrae: materia, titulo, contenido, tags

    # 1. Crear el apunte
    id_apunte = crearApunte(mirequest, session['id_usuario'])

    # 2. Si hay archivos, subirlos
    if id_apunte and 'archivo[]' in request.files:
        file_result = {}
        upload_file(file_result) # Sube archivos a static/uploads/

        for archivo_info in file_result.get('archivos', []):
            if not archivo_info.get('file_error'):
                crearMedia(id_apunte, archivo_info['file_name'],
                           archivo_info['file_name_new'])

    return id_apunte

```

4. Model inserta en BD (model.py)

```

def crearApunte(dic, idusuario):
    q = """
        INSERT INTO apunte (id_usuario, id_materia, head, content, tags,
        fechahora)
        VALUES (%s, %s, %s, %s, %s, %s)
    """
    val = (
        idusuario,
        dic.get('materia'),
        dic.get('titulo'),
        dic.get('contenido'),
        dic.get('tags'),
        datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    )
    id = insertDB_return_id(BASE, q, val)
    return id # Devuelve el ID del apunte creado

```

5. _mysql_db ejecuta SQL

```

def insertDB_return_id(configDB, sql, param):
    mydb = conectarBD(configDB)
    cursor = mydb.cursor()

    cursor.execute(sql, param) # Ejecuta INSERT
    mydb.commit() # Confirma cambios

```

```
id = cursor.lastrowid # Obtiene el ID auto-generado

cursor.close()
cerrarBD(mydb)
return id
```

6. Resultado

- Apunte guardado en tabla `apunte`
- Archivos guardados en tabla `media`
- Usuario redirigido a `/apunte?apunte=42`

Templates con Jinja2

Renderización de datos

controller.py:

```
param = {
    'apunte': {
        'titulo': 'Límites',
        'contenido': 'Definición...',
        'usuario': {'apodo': 'juanp'}
    },
    'comentarios': [
        {'apodo': 'martav', 'comentario': 'Genial!', 'fecha': '2025-01-01 12:00'}
    ]
}
return render_template('apunte.html', param=param)
```

templates/apunte.html:

```
<h1>{{ param.apunte.titulo }}</h1>
<p>Por: {{ param.apunte.usuario.apodo }}</p>
<div>{{ param.apunte.contenido }}</div>

<h2>Comentarios</h2>
{% for comentario in param.comentarios %}
<div class="comment">
    <strong>{{ comentario.apodo }}</strong>
    <p>{{ comentario.comentario }}</p>
    <small>{{ comentario.fecha }}</small>
</div>
{% endfor %}
```


Docker y Despliegue

Dockerfile

```
FROM python:3.9
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "run.py"]
```

docker-compose.yml

```
version: "3.8"
services:
  web:
    build: .
    ports:
      - "5000:5000"
    volumes:
      - ./static:/app/static
      - ./templates:/app/templates
    environment:
      - MYSQL_HOST=db
      - MYSQL_USER=root
      - MYSQL_PASSWORD=password
      - MYSQL_DATABASE=caaj

  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: caaj
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

Comandos:

```
docker-compose up -d --build # Construir y levantar
docker-compose logs web      # Ver logs del backend
docker-compose restart web    # Reiniciar Flask
```

Diagramas de Flujo

Flujo de Login

```
Usuario ingresa email/password
    ↓
route.py recibe POST /logging
    ↓
controller.py → crearSesion()
    ↓
model.py → obtenerUsuarioXEmailPass()
    ↓
_mysql_db.py → SELECT * FROM usuario WHERE email=%s AND pass=%s
    ↓
Si existe → cargarSesion() → redirect('/home')
Si no existe → return login con error
```

Flujo de Votación

```
Usuario click Like/Dislike
    ↓
fetch('/votar_apunte', {tipo: 'like'})
    ↓
controller.py → votar_apunte_process()
    ↓
model.py → votar_apunte_db()
    ↓
_mysql_db.py → INSERT/DELETE en tabla reaccion
    ↓
COUNT likes y dislikes
    ↓
return JSON {likes: 10, dislikes: 2, voto_actual: 'like'}
    ↓
JavaScript actualiza UI sin recargar
```

Configuración de la BD

appConfig.py:

```
BASE = {
    'host': 'localhost',
    'user': 'root',
    'pass': 'password',
    'dbname': 'caaj'
}
```

Conexión en _mysql_db.py:

```
def conectarBD(configDB):  
    mydb = mysql.connector.connect(  
        host=configDB.get('host'),  
        user=configDB.get('user'),  
        password=configDB.get('pass'),  
        database=configDB.get('dbname'),  
        charset='utf8mb4',  
        use_unicode=True  
    )  
    return mydb
```

Recursos y Comandos Útiles

Comandos SQL para debugging

```
-- Ver todos los apuntes con sus usuarios  
SELECT a.head, u.apodo, m.materia  
FROM apunte a  
JOIN usuario u ON a.id_usuario = u.id  
JOIN materia m ON a.id_materia = m.id;  
  
-- Contar votos de cada apunte  
SELECT a.head,  
       SUM(CASE WHEN r.valor='like' THEN 1 ELSE 0 END) as likes,  
       SUM(CASE WHEN r.valor='dislike' THEN 1 ELSE 0 END) as dislikes  
FROM apunte a  
LEFT JOIN reaccion r ON a.id = r.id_apunte  
GROUP BY a.id;  
  
-- Ver comentarios con usuarios  
SELECT a.head, u.apodo, c.content  
FROM comentario c  
JOIN apunte a ON c.id_apunte = a.id  
JOIN usuario u ON c.id_usuario = u.id;
```

Comandos Flask útiles

```
# Ver logs en tiempo real  
docker-compose logs -f web  
  
# Reiniciar sin rebuild  
docker-compose restart web  
  
# Acceder a la BD desde terminal  
docker-compose exec db mysql -uroot -ppassword caaj
```

Próximas Mejoras Recomendadas

1. Seguridad:

- Hashear contraseñas con `bcrypt`
- Validar inputs del lado del servidor
- Implementar CSRF tokens

2. Funcionalidades:

- Sistema de notificaciones
- Búsqueda por rango de fechas
- Exportar apuntes a PDF

3. Performance:

- Caché de consultas frecuentes
- Paginación de resultados
- Índices en columnas buscadas

4. UX:

- Editor Markdown con preview
 - Drag & drop para archivos
 - Dark mode
-

Troubleshooting Común

Error: "No module named mysql.connector"

```
pip install mysql-connector-python
```

Error: "Access denied for user"

Verifica credenciales en `appConfig.py` y que MySQL esté corriendo.

Archivos no se suben

- Revisar permisos de `static/uploads/`
- Verificar `MAX_CONTENT_LENGTH` en `upload_file()`

Sesión no persiste

- Verificar que `SECRET_KEY` esté configurada en `run.py`
-

Glosario

- **MVC:** Modelo-Vista-Controlador (patrón de arquitectura)
 - **ORM:** Object-Relational Mapping (no usado en este proyecto, queries directas)
 - **CRUD:** Create, Read, Update, Delete
 - **Session:** Almacenamiento temporal de datos del usuario (cookies cifradas)
 - **Template:** Archivo HTML con lógica Jinja2 para renderizar datos
 - **Route:** URL que mapea a una función del controlador
 - **Query:** Consulta SQL
 - **Fetch:** API de JavaScript para hacer peticiones HTTP
 - **AJAX:** Peticiones asíncronas sin recargar la página
-

¡Fin de la documentación! 🎉

Si tenés dudas específicas sobre alguna parte, revisá el código correspondiente o probá hacer queries en MySQL para entender mejor el flujo de datos.